

Faithful Backends for HE MeTTa

Bag Semantics, Candidate Selection, Co-Reference Preservation,
and Revision-Based Query Caching

Zar Goertzel
Oruži

March 2026

Abstract

This note sketches a correctness story for backend implementations of HE MeTTa. The central claim is that a practical backend should not be identified with the full runtime semantics of a MeTTa space. The semantics lives at the bag level: repeated derivations and repeated atoms matter. The backend may quotient that semantics to support-level structure for indexing, as long as the quotient and the refinement layers are explicit and proved faithful. Our current Lean development decomposes this story into three layers: lists of evaluator outcomes, multisets of semantic results, and finite support/index views. On top of this, we add a concrete candidate-selector architecture for PathMap tries, a counted-space refinement explaining why duplicate adds need not change trie support, an injective canonicalization discipline for variable identities, a variant-key query correctness layer for alpha-equivalent cache keys, and a revision-based cache model for query reuse. Together these results explain why CeTTa’s runtime should use PathMap for candidate narrowing, native matching for exact unification, explicit count metadata for bag semantics, variant-aware materialization for cached answers, and explicit invalidation rules for cached query answers.

1 Problem Statement

Backend bugs in MeTTa runtimes tend to fall into a small number of families:

1. candidate selectors that are too weak or too strong;
2. storage layers that silently collapse semantic multiplicity;
3. canonicalization passes that preserve alpha-shape but break co-reference;
4. query caches that reuse answers beyond the mutation boundary for which they were computed.

These failures are easy to confuse because they often show up through the same surface symptom: a query returns the wrong answer after spaces have been indexed, cloned, canonicalized, or cached. The right response is not to “prove the runtime” in one monolithic theorem. Instead we prove one obligation per runtime seam.

2 Semantic Layers

The first part of the development says that the semantic carrier for HE evaluation is not a set but a bag.

Evaluator layer. The runtime naturally produces a list of outcomes. This is the computational layer where order, scheduling, and implementation strategy still matter.

Semantic layer. `NondeterminismCarrier.lean` moves to `Multiset`, yielding the correct semantic equality for repeated results. `BagSupportBridge` then proves that support is the quotient visible to set-like indexing.

Index layer. `MonadMorphismChain.lean` and `SpaceQuerySupport.lean` package the resulting bridge:

$$\text{List} \rightarrow \text{Multiset} \rightarrow \text{Finset}.$$

This is the level at which a trie, hash table, or cached query store is allowed to operate.

Positive example. Adding the same atom twice should increase the bag count while leaving the support unchanged.

Negative example. Treating a set-valued index as if it were itself the full MeTTa space semantics.

3 Faithful Backend Contract

`SpaceQuerySupport.lean` gives the contract point:

`FaithfulBackend` means backend queries agree with HE queries at support level.

This is the right initial obligation for a backend such as a `PathMap`-backed space:

1. answer the same support-level equation queries;
2. answer the same support-level type queries;
3. refine support semantics with explicit count or row metadata, rather than burying multiplicity in trie keys.

The counted-space refinement in `CandidateArchitecture.lean` makes this concrete. `PathMap` stores support. The multiplicity layer stores how many copies exist. The two are intentionally separated.

4 Candidate Selection and PathMap

Theorems in `CandidateArchitecture.lean` now justify the architecture that CeTTa uses in practice:

1. the trie performs candidate selection;
2. native matching filters those candidates exactly;
3. the composition equals direct querying.

The abstract theorem `twoPhase_eq_direct` proves the shape of the argument. The concrete theorem `concrete_twoPhase_eq_direct` instantiates it for a trie-based selector. The prefix narrowing results then move beyond the unrealistic “scan all support” model and state the performance-relevant theorem: useful selectors may be strict support subsets without losing correctness, provided they are sound and phantom-free.

This is the strongest answer to the question “why not let the backend do all the unification?” Because candidate narrowing and exact matching are different obligations. PathMap is excellent at the first and should not be forced to simulate the second through a lossy bridge layer.

5 Co-Reference Preservation

The nastiest runtime bugs are usually not about support. They are about binders.

`CoReferencePreservation.lean` isolates the right invariant: variable canonicalization must be injective. Under an injective renaming:

1. repeated source variables stay repeated;
2. distinct source variables stay distinct;
3. composed renamings remain injective.

This is the formal obligation behind runtime components such as `term_universe.c` and the canonicalization logic in `table_store.c`. If a runtime stores

$$(\rightarrow (\rightarrow \varphi\psi)(\rightarrow \varphi\varphi))$$

and later materializes

$$(\rightarrow (\rightarrow \varphi\psi)(\rightarrow \chi\chi)),$$

then the bug is not “just alpha-equivalence.” It is a failure to preserve co-reference.

Recent work strengthens this seam in two directions. First, `BindingComposition.lean` exposes `Bindings.Extends` as the right invariant for seed-preserving matching. Second, `VariantQueryCorrectness.lean` proves a bisimulation theorem `simpleMatch_rename_bisim` using the fuel-free renaming `applyAtomTotal`. This proves that alpha-equivalent query keys do not merely preserve support; they preserve the right notion of matching success needed for variant-key cache reuse.

6 Revision-Based Query Caching

`CacheCorrectness.lean` already captures the main cache seam:

1. populate a cache entry from a space and a query;
2. reuse it only when the space revision still matches;
3. invalidate lazily on mutation;
4. preserve cache validity for non-crossing mutations such as duplicate `add-atom-nodup`.

The important point is that cache correctness is not merely a performance claim. It is part of backend faithfulness. A cache that returns stale support answers is semantically wrong even if the trie and matcher are otherwise correct.

The newer files `VariantQueryCorrectness.lean` and `BindingCacheSoundness.lean` extend this from support-level cache discipline to binding-aware reuse:

1. `variant_queries_same_rhs` proves that alpha-equivalent queries produce the same right-hand-side atoms after `queryEquations`;
2. `variant_cache_binding_shape` proves that the result shapes agree under variant reuse;
3. `cache_binding_exact` and related lemmas isolate the exact-versus-variant parts of the cache story.

The theorem family around mutation invalidation is especially important for implementations that mix:

1. candidate narrowing via a support index;
2. exact matching via native matcher state;
3. explicit count metadata;
4. snapshot or persistent clones.

7 Runtime Seam Map

Lean file	Runtime seam	Obligation
<code>CandidateArchitecture.lean</code>	<code>space_match_backend.c</code>	trie candidate selection + native exact rematch
<code>BagSupportBridge.lean</code>	counted-space layer	support/count split over bag semantics
<code>CoReferencePreservation.lean</code>	<code>term_universe.c</code>	canonical storage preserves variable identity
<code>VariantQueryCorrectness.lean</code>	<code>table_store.c</code>	variant-key queries preserve RHS answers
<code>BindingCacheSoundness.lean</code>	<code>table_store.c</code>	cached answer shapes remain binding-compatible
<code>BindingComposition.lean</code>	<code>space_match_backend.c</code>	seed bindings survive re-match/merge
<code>SeedRematchContract.lean</code>	<code>space_subst_match_with_seed.c</code>	seeded rematch extends seed, rejects loops, imported binding parity
<code>CacheCorrectness.lean</code>	<code>table_store.c</code>	revision-based query reuse and invalidation
<code>IncrementalTableSemantics.lean</code>	<code>term table_store.c</code>	partial table soundness, answer monotonicity, completion
<code>CeTTaRuntimeContracts.lean</code>	<code>term_canon.c, session.c</code>	canonicalization safety, rollback, effect/profile contracts
<code>SpaceQuerySupport.lean</code>	backend interface	support-level agreement target

8 What Belongs Here, and What Does Not

This paper should carry the full backend-correctness story:

1. bag semantics and support quotients;
2. counted-space refinement;
3. candidate-selection correctness;
4. **seeded native rematch** — the theorem that turns candidate parity into binding parity (`SeedRematchContract.lean`);
5. co-reference-preserving canonical storage;
6. revision-based cache correctness;
7. incremental tabling soundness (`IncrementalTableSemantics.lean`);
8. runtime contracts: canonicalization, rollback, effect classification (`CeTTaRuntimeContracts.lean`);
9. theorem-to-runtime seam mapping.

The seeded-rematch result is the key new contribution: CeTTa’s imported backend is binding-correct because candidate narrowing (from PathMap trie or exact match) is followed by seeded native rematch (`space_subst_match_with_seed`). The theorem `imported_seedRematch_binding_parity` proves that when imported candidates are sound, the two-phase query (candidates then rematch) produces the same result set as direct matching. Loop rejection (`seedRematchSafe_no_loop`) and seed extension (`seedRematch_extends_seed`) complete the safety story.

Enabling direct bridge bindings requires mapping each proved precondition to a specific CeTTa runtime check. The relevant fragments are ground-only, single-variable, and full V2; each has its own decode and binding obligations.

As of April 2026, ground-only is enabled in the runtime (tracked by `imported-v2-ground-skip`). The next rung, single-variable queries, is fully specified in Lean (`SingleVarSafe` in `ImportedRowContract.lean`) with conformance and negative tests, but remains gated pending bridge value decode verification. The Lean preconditions (`singleVarSafe_implies_rematchFreeSafe`, `ground_value_no_loop_risk`, `singleBinding_merge_decidable`) document exactly what must hold before a MORK implementer flips that switch.

By contrast, the dedicated PathMap formalization paper should keep the trie-specific material foregrounded:

1. algebraic hierarchy;
2. zipper structure;
3. concrete FTrie selector;
4. support/count and zero-crossing results insofar as they clarify trie usage.

Similarly, the CeTTa paper should use the results here to justify runtime architecture choices, rather than reproving the whole formal stack.

9 End-to-End Pipeline

The full formal pipeline is now connected across three Lean files:

Layer	File	Key Theorem
Bag semantics	BagSupportBridge.lean	support_bind
Trie storage	HEBridge.lean	same_support_same_trie
Selection	CandidateArchitecture.lean	concrete_twoPhase_eq_direct
Matcher instantiation	PathMapMatcherInstance.lean	he_twoPhase_eq_direct
Cache policy	CandidateArchitecture.lean	cache_policy_stable/growing/shrinking
Variant reuse	VariantQueryCorrectness.lean	variant_queries_same_rhs
Binding reuse	BindingCacheSoundness.lean	variant_cache_binding_shape
Contract	CandidateArchitecture.lean	BackendContract, backendContract_of_faithful
Pipeline	HEInterface.lean	end_to_end_pipeline

The `end_to_end_pipeline` theorem bundles all three properties in one statement: given a faithful trie, (1) query correctness holds, (2) no phantom candidates exist, and (3) duplicate adds preserve support.

The `BackendContract` structure serves as the single proof obligation for any `PathMap`-backed space: provide `trieFaithful` (every support atom has its encoding in the trie), and the full contract follows automatically via `backendContract_of_faithful`.

The cache invalidation interface classifies mutations into three kinds:

- `supportStable`: no cache invalidation needed (duplicate adds, high-count removes);
- `supportGrowing`: invalidate queries matching the new atom;
- `supportShrinking`: invalidate queries matching the removed atom.

Each maps to one branch in `table_store.c`.

10 Current Status and Next Proof Obligations

The current development already supports the following claims:

1. support-level backend agreement is the correct first target;
2. `PathMap` candidate selection plus native matching is a correct architecture;
3. duplicate adds do not need to mutate trie support;
4. injective canonicalization is the right shape for binder hygiene;
5. alpha-equivalent query keys preserve right-hand-side answers and result shape under cache reuse;
6. revision counters justify lazy cache invalidation.

The most urgent next formal obligations are:

1. storage/materialization preserves query answers, not just variable injectivity;

2. snapshot/clone operations preserve backend faithfulness;
3. revision and snapshot semantics are connected explicitly enough to support staged or open table entries.

These are the theorems most likely to help with the remaining nasty binder bugs in the current CeTTa runtime work.

The integration contract must connect each proved theorem to its runtime implementation (`space.c`, `table_store.c`, `space_match_backend.c`, `term_universe.c`) and the corresponding regression test target. A theorem alone does not establish that a particular runtime path satisfies its premises.